

Interactive Smart Contracts for Secure and Trustworthy Online Content Consumption Tracking

Shubham Joshi, Dillon Davidson, Zuobin Xiong, and Junggab Son*

Abstract—The rapid evolution of the digital landscape has made content creation and distribution more accessible than ever. However, ensuring the integrity and accuracy of content consumption and download tracking remains a significant challenge. Centralized tracking systems often lack transparency and are susceptible to security breaches, raising concerns for both content creators and service providers. To address these issues, we propose a blockchain-based solution that leverages smart contracts for tamper-proof content tracking. Our approach introduces two secure and interactive smart contracts: an Elliptic Curve Cryptography-based Proxy Re-Encryption smart contract (ECC-PRE-SC) and a Counting smart contract (Count-SC). The ECC-PRE-SC facilitates secure re-encryption, ensuring that only authorized entities can access the encrypted content key. Meanwhile, the Count-SC maintains an immutable and verifiable record of content consumption and downloads. Content Owners (COs) deploy the ECC-PRE-SC, embedding a list of authorized service providers' wallet addresses to enforce access control and prevent unauthorized modifications to download counts. Each ECC-PRE-SC instance is linked to a dedicated Count-SC, ensuring precise and independent tracking for specific content. Our security analysis verifies the system's robustness against potential threats, while its successful deployment on the Sepolia Testnet demonstrates its feasibility. This research enhances trust and transparency in digital content distribution, providing significant benefits to content creators, service providers, and end users.

Index Terms—Content Consumption Tracking, Interactive Smart Contract, Online Content Sharing, Trustworthiness, Blockchain.

I. INTRODUCTION

CONTENT creation is evolving rapidly in today's fast-paced digital world. The widespread availability of the Internet has made it easier than ever to share content and reach larger audiences [2]. However, one significant challenge for Content Owners (COs) is ensuring secure distribution [3]. To address this, cloud-sharing platforms, referred to as Service Providers (SPs), have emerged, helping COs to grow their user base while maintaining security. This ecosystem benefits both COs and SPs, as the success and profitability of content often depend on its popularity, typically measured by the number of times it is consumed or downloaded [4]. Therefore, accurate and tamper-proof records of content consumption are essential for both parties.

Traditional management of digital rights for online media still faces numerous challenges [5]. A critical issue lies in

ensuring the accuracy and tracking of the count for content consumption or downloads. In most cases, SPs manage these count through proprietary software, which they can modify at will. Although blockchain-based counting systems exist in other domains, they commonly rely on off-chain reporting or oracle consensus and are therefore insufficient for guaranteeing secure and transparent consumption counts in DRM settings (see Section II-A). This creates a trust deficit between COs and SPs, as their profitability hinges on transparent and accurate reporting of content consumption [6].

Many Digital Rights Management (DRM) systems are currently used to manage content rights, facilitating the transfer of these rights from COs to users via intermediaries. These DRM systems are typically centralized, controlled, and operated by entities, often SPs, focused on distributing content to a broader audience. A key advantage of this centralized approach is its ability to protect COs by safeguarding their intellectual property [7]. However, there are risks involved, as third parties or other entities may violate established rules and engage in harmful activities, such as distributing content to unauthorized users, which undermines the interests of COs.

Moreover, if a third-party server is compromised, it can result in sensitive digital content information being leaked. Disruptions to server access can also impact all users who depend on the service, highlighting the vulnerabilities of centralized DRM systems. The centralized nature of current DRM systems makes them susceptible to single points of failure and cyberattacks. They also lack the tamper resistance and distributed trust offered by decentralized architectures, such as blockchains [8]. Furthermore, managing content rights and usage policies across multiple stakeholders in the content distribution ecosystem is hindered by limited flexibility and transparency [9], [10].

To address the challenge of maintaining integrity and ensuring a tamper-proof count of content consumption or downloads, we propose a system that leverages two secure smart contracts: an Elliptic Curve Cryptography-based Proxy Re-Encryption Smart Contract (ECC-PRE-SC) and a Counting Smart Contract (Count-SC). The ECC-PRE-SC manages the re-encryption process as part of granting access to content and also acts as a verifier for the Zero Knowledge Proof process [11], while the Count-SC records the number of times the ECC-PRE-SC is executed successfully.

The CO deploys the ECC-PRE-SC, embedding the wallet addresses of authorized SPs and the encrypted content key to facilitate re-encryption operations. When the constructor of ECC-PRE-SC is invoked, it creates a new instance of the Count-SC within the ECC-PRE-SC contract. This triggers the

S. Joshi, D. Davidson, Z. Xiong, and J. Son are with Department of Computer Science, University of Nevada, Las Vegas, Las Vegas, Nevada, USA. Email: {shubham.joshi, junggab.son@unlv.edu}

A preliminary version of this work was presented at SecureComm 2024 and was published in its proceeding [1]

*Corresponding Author

deployment of the Count-SC, with the ECC-PRE-SC's address designated as the sole authorized entity allowed to execute the increment function in the Count-SC. Furthermore, This design ensures that the count can only be modified through the increment function, which must be called by an authorized entity.

The proposed system effectively addresses the challenges of ensuring the integrity of the content consumption count. By leveraging smart contracts, we provide a solution that enables tamper-proof tracking and recording of content downloads or usage. This approach significantly improves trust and transparency between *COs* and *SPs* throughout the content distribution process. Furthermore, the proposed scheme contributes to the advancement of decentralized architectures, such as blockchain, providing a more robust alternative to traditional centralized DRM systems. The outcomes of our research benefit not only *COs* and *SPs* but also users who rely on secure and transparent platforms for content distribution.

The main contributions of this paper can be summarized as follows:

- To the best of our knowledge, no prior DRM work cryptographically enforces accurate consumption/download counts. Although blockchain counting exists in other domains, it relies on oracles or off-chain reporting and is therefore insufficient for secure and transparent counts. Recognizing this gap led to the development of the solution presented in this paper.
- An interactive smart contract architecture has been proposed that ensures the integrity of content consumption count. One contract handles re-encryption using an ECC-based Proxy Re-encryption Scheme, while the other tracks and secures consumption or download counts.
- A security proof based on a defined threat model has been provided that shows that our system is secure. It is resilient to adversaries trying to modify the count in the Count-SC or gain unauthorized access to content keys through re-encrypted content keys on the blockchain.
- The feasibility of the proposed architecture is demonstrated via implementation on Sepolia Testnet. The implementation also shows consistent gas consumption during re-encryption, even with varying content key lengths in the smart contract.
- An efficient version of the smart contract was developed, which improves the usage of gas by approximately 63% from our previous version.

The remainder of this paper is organized as follows. Section II reviews related work on encryption schemes, smart contract-based DRM, and cloud content sharing. Section III covers preliminaries, laying the foundation for our system. Section IV defines the problem through the system model, threat model, and design goals. Our proposed scheme, including two key smart contracts, is presented in Section V. Section VI validates the correctness of the scheme, while Section VII provides a formal security analysis. Section VIII discusses implementation and evaluation, and Section IX concludes with a summary of our contributions.

II. RELATED WORK

This section explores recent research trends in secure content sharing within cloud environments. We specifically examine studies focused on encryption schemes, cloud content sharing mechanisms, and the application of smart contracts within DRM systems.

A. Blockchain and Counting in Other Domains

Blockchain-based counting mechanisms span several domains, including supply chains, advertising, and voting. In supply chains, platforms such as IBM Food Trust ensure immutable traceability of goods, capturing both provenance and quantitative data [12]. Westerkamp et al. introduced token recipes to formalize manufacturing transformations, preserving quantity conservation [13], while bipartite-matching frameworks provides one-to-one tracking of product flow [14]. In advertising, B²DAM immutably logs campaign exposures [15], whereas CountChain provide transparent exposure reporting by recording campaign events and validating off-chain impressions [16], though these approaches ultimately depend on oracle consensus or external reporting rather than cryptographic enforcement.

In voting and provenance management, blockchain functions as a tamper-resistant ledger that enhances transparency and auditability. Hyperledger-based voting systems with Shamir secret sharing and IoT-integrated e-polling frameworks focus on securing ballots and ensuring correct tallying [17], [18]. Immutable logging solutions, such as Ozdayi et al.'s cross-site access auditing [19], leverage blockchain to provide verifiable audit trails, where counting arises through aggregation of logs rather than cryptographic coupling of events to counters. Collectively, these efforts highlight blockchain's capacity to ensure trust and accountability in contexts where tallying is central, with varying reliance on on-chain mechanisms versus off-chain validation.

These prior efforts illustrate blockchain's suitability for transparent, tamper-proof record-keeping across domains such as supply chain, advertising, voting, and logging. However, they differ fundamentally from our approach: in those systems, counters increment based on reported events or oracle validation, whereas in our design, a counter update is only possible as a by-product of a zero-knowledge-verified proxy re-encryption that yields a usable content key. Thus, content consumption counts are cryptographically guaranteed to correspond to genuine access events, not merely asserted or reported.

B. Cloud Content Sharing

Several studies address the challenges of secure cloud content sharing. Zhang et al. introduced CREAMS, a scheme prioritizing copyright protection and data privacy within media sharing contexts [20]. Their design combines additive homomorphic proxy re-encryption with asymmetric fingerprinting techniques to address the problem of data privacy leakage, infringement on the data owner's copyright.

Feng et al. proposed a proxy re-encryption scheme combining attribute-based and identity-based encryption, integrating blockchain to prevent data tampering and eliminate reliance on trusted third parties [21]. Liu et al. improved efficiency by removing bilinear pairs and addressing certificate management and key escrow issues through elliptic curve groups, certificate-based encryption, and identity-based proxy re-encryption [22]. Both schemes enhance security and efficiency while preventing unauthorized data exposure.

Wang et al. proposed a blockchain-based secure data-sharing scheme for the Industrial Internet of Things (IIoT), enabling anonymous cross-domain access for authorized devices [23]. By combining broadcast encryption and proxy re-encryption, their scheme ensures secure, flexible data sharing while preserving privacy, though re-encryption relies on a cloud-based trusted third party.

Prior cloud content-sharing schemes emphasize confidentiality, policy-driven access, and privacy—via homomorphic or attribute/identity-based proxy re-encryption, watermarking/fingerprinting, and occasionally blockchain for provenance, but they do not cryptographically ensure that usage reports equal genuine accesses. In particular, they (i) leave consumption counting to mutable platform software, (ii) lack a mechanism that forces a counter increment to be an unavoidable consequence of delivering a real, decryptable key, and (iii) sometimes retain a trusted re-encryption proxy, re-introducing unilateral control.

C. DRM based on Smart Contracts

The potential of blockchain and smart contracts in DRM systems has been demonstrated in several studies. Zhang et al. addressed limitations of traditional centralized DRM by employing smart contracts for reliable copyright transactions [24]. Garba et al. proposed a distributed media transaction framework combining digital watermarking with a scalable blockchain model to secure authorized access, protect offline content, and enable copyright reclamation in case of leakage [25].

Gao et al. introduced a blockchain-based DRM scheme using Multi-Authority Ciphertext-Policy Attribute-Based Encryption (MA-CPABE) and proxy re-encryption to enhance confidentiality and support efficient copyright transfers [26]. Likewise, Kemmoe et al. leveraged smart contracts for secure, asynchronous group key exchange, improving flexibility in rights management [27].

While these works highlight blockchain's potential for DRM and secure key exchange, they do not enforce verifiable consumption counts and often rely on honest reporting. Our approach addresses this gap by coupling proxy re-encryption with a zero-knowledge-verified counter: the Count-SC increments only as a by-product of successful re-encryption in the ECC-PRE-SC, ensuring tamper-proof and auditable tracking. This eliminates unilateral trust in service providers and introduces immutable, cryptographically enforced usage verification, overcoming critical shortcomings in prior DRM frameworks.

D. Encryption Schemes for Secure Data Sharing

Duan et al. proposed an attribute-based proxy re-encryption scheme that supports re-encryption, access delegation, and blockchain-based logging through smart contracts [28], but it does not secure usage counts. Hussain et al. introduced a lightweight certificate-based signcryption with proxy re-encryption (CBSRE) using hyperelliptic curves for IoT-enabled smart grids [29]. Similarly, Agyekum et al. combined identity-based encryption with PRE to enhance IoT access control [30], while Keshta et al. integrated blockchain with PRE via the SM2 algorithm to improve data privacy [31]. Shabisha et al. explored ECC-based signcryption and PRE techniques to strengthen cloud data storage against curious providers [32].

These schemes collectively provide robust confidentiality, flexible access delegation, and efficient proxy re-encryption. However, none cryptographically bind re-encryption operations to an immutable counter or prevent service providers from misreporting usage. As a result, they leave consumption accountability dependent on platform honesty or off-chain mechanisms.

In contrast, our approach integrates ECC-based proxy re-encryption with a blockchain-resident counting contract. Each valid content access or data retrieval unavoidably and verifiably increments the counter. By embedding authorized provider addresses within the ECC-PRE-SC and employing zero-knowledge proofs to gate re-encryption, our scheme eliminates trusted intermediaries and ensures tamper-resistant, auditable consumption tracking, addressing critical accountability and transparency gaps in prior work.

III. PRELIMINARIES

This section introduces important preliminaries that are the building blocks of our proposed system.

A. Blockchain and Smart Contracts

Blockchain is a sequentially linked data structure operating in a peer-to-peer network with a reliable consensus mechanism, ensuring security, traceability, decentralization, and trustlessness. As a distributed ledger, it records verified transactions with timestamps, making data immutable and publicly verifiable. These features enable applications that function without trusted intermediaries [33] [34].

Smart contracts are decentralized programs that automate transactions between parties without a central authority [35]. Various platforms support smart contract development, including Bitcoin [36], Ethereum [37], Hyperledger Fabric [38], and Stellar [39], with Ethereum being the most widely adopted due to its stability and security [40].

B. Hardness Assumption

Our scheme's security is based on the following defined hard problems. First, we introduce the concept of elliptic curves. An elliptic curve $E(\mathbb{F}_q)$ is defined over a finite field $\mathbb{F}_q$. It has a base point generator $P \in E(\mathbb{F}_q)$ of prime order q . Together with the infinite point, all points on $E(\mathbb{F}_q)$ form

an additive group G . Given this, we consider a Probabilistic Polynomial-Time (PPT) adversary $\mathcal{A}$ and define the following hard problems:

1) *Discrete Logarithm Problem (DLP)*: : Let $(x, pk) \leftarrow \text{Gen}(1^\lambda, P)$ be a function that takes as inputs a security parameter 1^λ and the base point P , then outputs $x \in \mathbb{Z}_n^*$ and $pk = x \cdot P \in E(\mathbb{F}_q)$. Given $1^\lambda, pk$, and P as inputs, the probability that $\mathcal{A}$ outputs x' such that $x' = x$ should be negligible.

$$\Pr[x' \leftarrow \mathcal{A}(1^\lambda, pk, P) : x' = x] \leq \text{negl}(\lambda)$$

2) *Computational Diffie-Hellman (CDH) Problem*: : Let G be a large prime elliptic curve group of order q and P a generator of the group G . Then, the CDH problem on group G is defined as follows: Given $P, aP, bP \in G$, compute $abP = P \in G$ for any $a, b \in \mathbb{Z}_q^*$. The probability that any probabilistic polynomial-time (PPT) algorithm, denoted as $\mathcal{A}_{CDH}$, that can effectively solve the Computational Diffie-Hellman (CDH) problem is negligible:

$$\Pr[\mathcal{A}_{CDH}(G, q, P, aP, bP) = abP] \leq \text{negl}(\lambda)$$

C. Elliptic Curve based Proxy Re-encryption

Proxy Re-Encryption (PRE) allows a proxy to convert an encrypted message for one recipient into a format decryptable by another, without revealing the sender's secret key. For example, Alice can authorize a proxy to re-encrypt her emails for Bob without sharing her key. The algorithms needed for the PRE schemes are briefly outlined below:

- **Setup**(1^λ): This takes (λ) as security parameter and generates public parameters $params$.
- **KeyGenerate**($params$): This takes parameters, $params$, as input and returns the private key, sk_i , and corresponding public key, $pk_i = (pk_{i1}, pk_{i2})$.
- **Encrypt**($k_i, id_{CO}, pk_{CO}, params$): This takes the content key, k_i , identity of the CO, id_{CO} , public key of the CO, pk_{CO} , and public parameters, $params$, as input and returns the encrypted content, $C = (C_1, C_2, C_3, C_4, C_5)$.
- **ReKeyGenerate**($c, params, id_{CO}, id_{U_i}, pk_{CO}, pk_{U_i}$): This takes a ephemeral randomness, c , public parameters, $params$, identity of the CO, id_{CO} , public key of the CO, pk_{CO} , identity of the User, id_{U_i} , and public key of the User, pk_{U_i} , as input and returns a re-encryption key, $rk_{A \rightarrow B} = (rk_1, rk_2, rk_3)$.
- **Re-Encrypt**($C, rk_{A \rightarrow B}, params$): This takes the encrypted content, C , re-encryption key, $rk_{CO \rightarrow U}$, and public parameters, $params$, as input, and it returns the re-encrypted content, $C' = (C'_1, C'_2, C'_3, C'_4)$.
- **Decrypt**($C, id_{CO}, sk_{CO}, params$): This takes the encrypted content, C , identity of the CO, id_{CO} , private key of the CO, sk_{CO} , and public parameters, $params$, and it returns the decrypted content, k_i .
- **Re-Decrypt**($C', id_{CO}, pk_{CO}, id_{U_i}, sk_{U_i}, params$): This takes the re-encrypted content, C' , identity of the CO, id_{CO} , public key of the CO, pk_{CO} , identity of the User, id_{U_i} , private key of the User, sk_{U_i} , and public parameters, $params$, as input and returns the decrypted content, k_i .

IV. PROBLEM DEFINITIONS

A. System Model

Fig. 1 illustrates the proposed system model. The model considers a scenario with a CO and a single blockchain network, reflecting a realistic setting where CO want to distribute their content through various SPs. It also considers multiple users who can interact with SPs to access content. The model consists of five entities: Trusted Third Party (TTP), Content Owners (COs), Service Providers ($SP_1, SP_2, \dots, SP_n$), Blockchain Network (BN), and Users ($U_1, U_2, \dots, U_n$).

1) *Trusted Third Part (TTP)*: The TTP is responsible for setting up the public parameters and generating key pairs for the COs, SPs, and users. It securely distributes the public keys and private keys to the respective parties.

2) *Content Owner (COs)*: The CO produces content like music, videos, and ebooks, encrypts it with a unique content key, and distributes it via SPs. The CO generates the content key which is used to encrypt the content. Furthermore, the CO deploys two smart contracts in the blockchain with the encrypted content key and authorized SPs wallet addresses. For every content, a unique content key is used for encryption, and consequently, a distinct smart contract is deployed for each.

3) *Service Providers (SPs)*: SPs receive encrypted content from COs and distribute it via their platforms. They generate re-encryption keys and interact with ECC-PRE-SC to obtain re-encrypted content keys, which are then forwarded to users along with the encrypted content.

4) *Blockchain Network (BN)*: BN consists of two smart contracts: ECC-PRE-SC, which verifies SPs using zero-knowledge proofs and re-encrypts content keys, and Count-SC, which tracks content consumption. As a decentralized ledger, it ensures tamper-proof data for COs and SPs.

5) *Users*: Users register with SPs to access encrypted content. After authentication, a user U_i receives the encrypted content and a re-encrypted key, which can only be decrypted with their private key. This allows them to retrieve the content key and decrypt the content securely.

B. Threat Model

The following models represent the list of attacks that may target to our system model.

- Threat Model 1: An adversary may analyze multiple re-encrypted content keys on the blockchain to deduce the content key. By locating transaction details via transaction hash, block hash, or smart contract address, they may attempt to infer the key and decrypt the content.
- Threat Model 2: A CO or SP may act as an adversary and attempt to maliciously manipulate the count recorded on the blockchain, seeking to change the count for personal gain. An unethical attempt to gain control over the Count-SC may be made in order to achieve this goal.
- Threat Model 3: A User with legitimate access to one content may attempt to leverage this access to gain unauthorized access to a different content. A User with a legitimate content key to one of the contents may

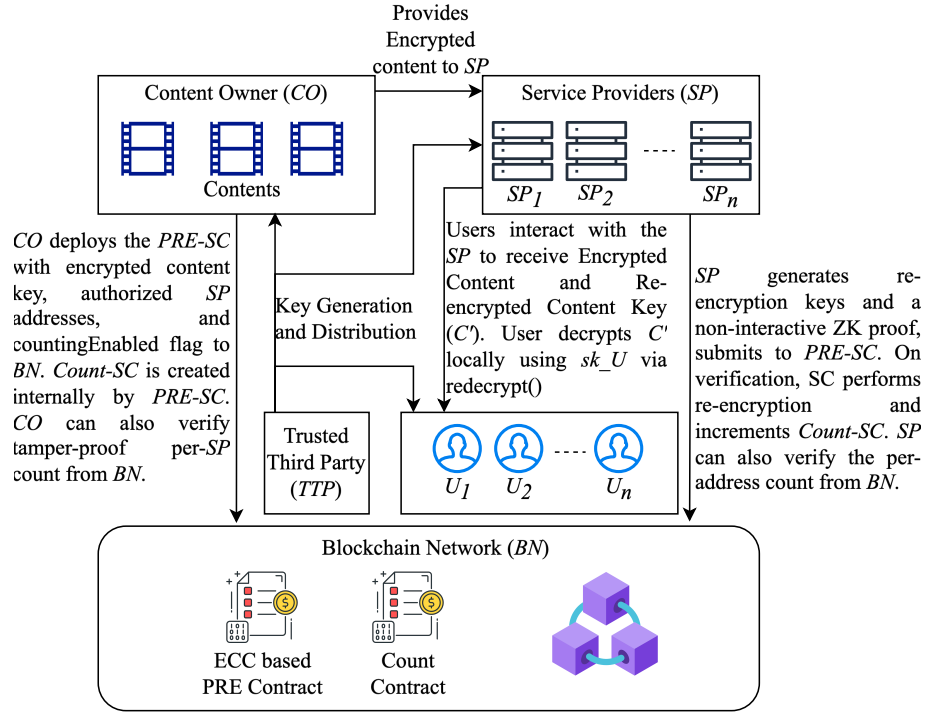


Fig. 1: System Model

maliciously try to decrypt other contents using the same key.

- Threat Model 4: The SP may act as an honest but curious entity seeking to retrieve information through the transactions that go through it. The SP intermediates re-encryption key creation and forwards encrypted and re-encrypted content to Users. The SP may attempt to obtain information regarding the content key through this process.
- Threat Model 5: An adversarial entity may attempt to impersonate an honest SP to request the re-encrypted content key from the ECC-PRE-SC or deliberately overload the ECC-PRE-SC to disrupt its functionality.

C. Design Goals

Considering the described system model and threat model, the following design goals has been established.

- The system aims to record and display the tamper-proof count of the number of content consumed or downloaded.
- The system aims to implement privacy-preserving techniques to anonymize or hide information that can otherwise be seen on the blockchain.
- The system aims to follow key independence, ensuring that there are different content keys for different contents, and they must be random during key establishment.

V. CONSTRUCTION OF THE PROPOSED SYSTEM

This section provides details of our proposed scheme and its components. It details the Elliptic Curve based Proxy Re-Encryption scheme as well as the design of the smart contracts. Table I defines notations used in this paper.

We propose the application of an Elliptic Curve Cryptography-based Proxy Re-Encryption scheme comprising polynomial-time algorithms. The first two algorithms are essential for key generation: Setup algorithm and Key Generation Algorithm. The TTP sets up the public parameters

TABLE I: Description of Notations

Notation	Description
pk_{CO} & sk_{CO}	Public key & Private key of CO.
pk_{U_i} & sk_{U_i}	Public key & Private key of User.
id_{CO} & id_{U_i}	Identity of CO & Identity of User.
q	Prime order.
G	Cyclic Additive group.
P	Generating Element of G .
$rk_{CO \rightarrow U}$	Re-encryption key generated by SP and sent to the ECC-PRE-SC for re-encryption process.
C_m	Encrypted content, $E_{k_i}(m)$.
C	Encrypted Content Key by CO, $E_{pk_{CO}}(k_i)$
C'	Re-encrypted Content Key by SC1.
m	Original or unencrypted content.
k_i	Original key used to encrypt the original content.
i, o	Large Prime Numbers.
v, j	Secret Random Integer and Random Number generated by SP in ZKP.
α, γ	Challenge and Proof generated by SP in ZKP.

using the Setup algorithm and then uses the Key Generation algorithm to generate the public and private keys for the COs, the SPs, and the Users. Aside these key, the symmetric content key, which is used to encrypt the content, is generated by the CO. The algorithm for the generation of k_i is not related to our proposed scheme, so we do not delve into it. The following outlines the Setup and Key Generation processes:

A. Setup and Key Generation

The preliminary steps required before executing the schemes are outlined in this subsection.

Setup: While inputting 1^λ , the security parameter, generate the public parameters set, $params$, which is given as follows:

- A prime number q with b bits is selected to create a cyclic additive group. This group, denoted as G , consists of elliptic curves whose order is the large prime q , with P serving as the generator element of G .

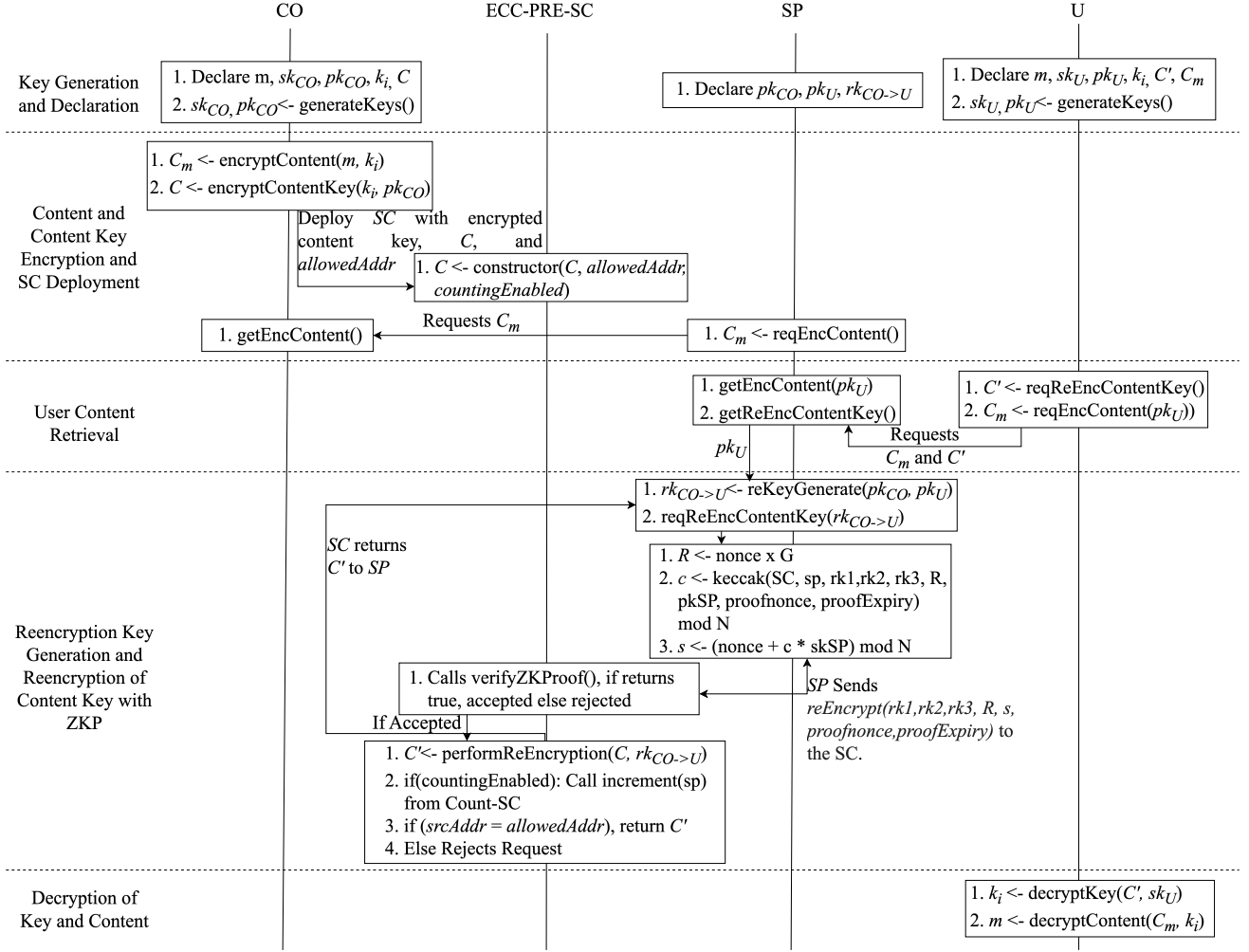


Fig. 2: Overall Process of the System reflecting ECC-PRE-SC

- b) Four hash functions are selected, with n and l representing the lengths of the random bit string utilized by the plaintext and Encrypt algorithm, respectively.

$$H_1 : \{0, 1\}^n \times \{0, 1\}^l \times \{0, 1\}^* \times G^2 \rightarrow \mathbb{Z}_q^*$$

$$H_2 : G \rightarrow \{0, 1\}^{n+l}$$

$$H_3 : G \times G \times \{0, 1\}^{n+l} \times G \rightarrow \mathbb{Z}_q^*$$

$$H_4 : \{0, 1\}^* \times \{0, 1\}^* \times G \rightarrow \mathbb{Z}_q^*$$

- c) The set of public parameters is output as: $params = \{G, q, P, n, l, H_1, H_2, H_3, H_4\}$.

Key Generation: *TTP* generates the public and private key pairs for the *COs*, *SPs*, and users using the KeyGenerate algorithm. While inputting the public parameters $params$, this algorithm randomly chooses $sk_i = x_i \in \mathbb{Z}_q^*$ as the private key for User i , and calculates the public key $pk_i = (pk_{i1}, pk_{i2}) = (x_i P, y_i P)$. It then outputs the private key sk_i and the public key (pk_{i1}, pk_{i2}) for User i .

B. Encryption and SC Deployment

The following step involves encrypting m and k_i , which is used to encrypt the m and then deploying the smart contract to the blockchain network.

The algorithms of *CO* consist of functions $encryptContent()$, $encryptContentKey()$ and $deployContract()$.

- $encryptContent(m, k_i)$: *CO* uses its content key, k_i , to encrypt the original content, m , and returns an encrypted content C_m .
- $encryptContentKey(k_i, pk_{CO}, id_{CO}, params)$: *CO* uses its public key, pk_{CO} , identity, id_{CO} , and $params$ to encrypt the content key, k_i , that was used to encrypt the content, and returns an encrypted key, C .

The encryption algorithm for encrypting m is not discussed in the paper, and it is assumed to use a symmetric key. This paper focuses on encrypting k_i . The algorithm used for the encryption of the k_i is given below:

Encrypt: While inputting the content key, $k_i \in \{0, 1\}^n$, the public key $pk_{CO} = (pk_{CO1}, pk_{CO2})$, the identity id_{CO} , and $params$, the *CO* does the following:

- Select a random ephemeral randomness $c \in \mathbb{Z}_q^*$.
- Select an l -bit $\delta \in \{0, 1\}^l$ randomly, and compute $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$, $f = c \cdot r$.
- Compute the ciphertext for the content key, $C_1 = rP$, $C_2 = c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1}$, $C_3 = (k_i || \delta) \oplus H_2(fQ_A)$, where $Q_A = P \cdot (pk_{CO1} + pk_{CO2})$.

- d) Select $t \in \mathbb{Z}_q^*$ randomly, and remaining ciphertext is computed, $C_4 = tP$, $C_5 = t + c \cdot r \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1} \cdot H_3(C_1, C_2, C_3, C_4)$.
- e) Original ciphertext for the content key is outputted, $C = (C_1, C_2, C_3, C_4, C_5)$.

After the encryption of k_i , C is deployed in the ECC-PRE-SC. CO executes the *deployContract* functions to deploy the SC with C and *allowedAddr*. The overall process of the system reflecting ECC-PRE-SC is shown in Fig. 2.

- a) *deployContract*($C_1, C_2, C_3, C_4, C_5, allowedAddr$): C and *allowedAddr* is set when the ECC-PRE-SC is deployed by the CO . The encrypted content key (C_1, C_2, C_3, C_4 , and C_5) are kept immutable as read-only, while hashed SP addresses in the contract obscure their original values.

Furthermore, the C_m is retrieved by SP from CO by calling the *getEncContent* function.

C. User Content Retrieval

User after the valid authentication process can request the content from the SP . The function, *reqKeyContent*(pk_{U_i}), requests the re-encrypted content key and encrypted content from SP while sending its public key, pk_{U_i} , and receives C_m and C' in return. This process invokes the process of creation of the re-encryption key by SP and subsequently the re-encryption process in the SC.

D. Re-encryption Key Generation and Re-encryption of Content Key with ZKP

SP generates the re-encryption key and ECC-PRE-SC performs the re-encryption process. The first process is to generate the re-encryption key, challenge, and proof for the re-encryption process. This is done by the SP . SP consist of *reqEncContent*(), *reKeyGenerate*(), *getReEncContentKey*(), *computeCommitment*(), *computeChallenge*(), and *computeProof*() functions.

- a) *reqEncContent*(): This function requests encrypted content, C_m , from the CO and stores it.
- b) *reKeyGenerate*($c, id_{CO}, pk_{CO}, id_{U_i}, pk_{U_i}$): This function generates a re-encryption key using $c, id_{CO}, pk_{CO}, id_{U_i}, pk_{U_i}$ and returns $rk_{CO \rightarrow U}$.
- c) *getReEncContentKey*($rk_{CO \rightarrow U}$): This function requests the re-encrypted content key from the ECC-PRE-SC, by sending $rk_{CO \rightarrow U}$, and receives C' , in return from the ECC-PRE-SC.
- d) *computeCommitment*(): This function computes A , and B , by taking a random value j such that $A = g^j(modw)$, and $B = g^j(modw)$, where g and w are generator elements, which belongs to a cyclic group and a large random prime number respectively.
- e) *computeChallenge*(): This functions computes the challenge, α , by concatenating g, h, yy, zz, A and B and using keccak256 hash function.
- f) *computeProof*(): This function computes the proof, γ , such that $\gamma = j - \alpha \cdot (modw)$.

The algorithm for the generation of the re-encryption key is as follows:

ReKeyGenerate: While inputting the public parameters *params*, $c, pk_{CO}, id_{CO}, pk_{U_i} = (pk_{U1}, pk_{U2})$, and id_{U_i} , this algorithm performs as:

- a) Calculate $s = H_4(id_{CO}, id_{U_i}, pk_{U_i})$.
- b) Then, compute $rk_1 = s^{-1} \cdot c \cdot pk_{CO1}$, $rk_2 = s^{-1} \cdot c \cdot pk_{CO2}$, $rk_3 = s^{-1} \cdot (pk_{CO1} + pk_{CO2})$.
- c) The proxy re-encryption key $rk_{A \rightarrow B} = (rk_1, rk_2, rk_3)$ is outputted.

The second process is re-encryption. An SP can initiate the re-encryption process with ECC-PRE-SC only after successfully completing zero-knowledge proof (ZKP) verification. Once $rk_{A \rightarrow B}$ is created, the SP (prover) embeds a secret message in a commitment and sends α and γ to ECC-PRE-SC (verifier). ECC-PRE-SC computes a challenge α' and compares it with α . If they match, verification succeeds, and re-encryption proceeds. ECC-PRE-SC then re-encrypts C using $rk_{A \rightarrow B}$ and returns C' to the SP . The contract includes *reEncrypt*(), *performReEncryption*(), and *verifyProof*(), as outlined in Fig. ??.

- a) *ReEncrypt*($i, o, yy, zz, w, \gamma, \alpha, rk_1, rk_2, rk_3$): This functions, first invokes the *verifyProof*() function for the Zero Knowledge Proof verification. If correct SP is communicating with the SP *verifyProof*() returns true and it proceeds further. Then, *performReEncryption*() is called for the actual re-encryption process. This function re-encrypts the C and returns C' to the SP .
- b) *verifyProof*($i, o, yy, zz, w, \gamma, \alpha$): This function takes i, o, yy, zz, w, γ and α as input and calculates a new commitment and α' . Then it compares the challenge sent by the SP with the newly derived challenge. If both are equal, it returns true; otherwise, it returns false.
- c) *performReEncryption*(rk_1, rk_2, rk_3): The ECC-PRE-SC uses the ($rk_{CO \rightarrow U} = rk_1, rk_2, rk_3$) sent by the SP to re-encrypt the C , which is already in the deployed smart contract, using this function. It then returns a re-encrypted content key C' to the SP .

The re-encrypt algorithm to generate the re-encryption keys is as follows: **Re-Encrypt:** While inputting the $C, rk_{CO \rightarrow U_i}$, and *params*, re-encryption occurs as follows:

- a) If $C_5P = C_4 + H_3(C_1, C_2, C_3, C_4)C_2$, then continue; otherwise, output $\perp$.
- b) Compute $C'_1 = rk_1 \cdot C_1$, $C'_2 = rk_2 \cdot C_1$, $C'_3 = C_3$, $C'_4 = rk_3 \cdot C_1$, and output a new re-encrypted ciphertext of the content key $C' = (id_{CO}, C'_1, C'_2, C'_3, C'_4)$.

Just before the C' , is generated, the increment function of the Count-SC is invoked. The process that occurs in Count-SC is shown in Fig. 3. The Count-SC verifies whether the request to the increment function originates from the address of the ECC-PRE-SC that created it. Once it is verified, Count-SC records the number of times the content is accessed or downloaded. Count-SC, is for counting purposes, which is deployed at the same time ECC-PRE-SC is deployed. Count-SC, as shown in Fig ??, has four functions:

- a) *constructor()*: It initializes the contract with the owner's address, that is the address of ECC-PRE-SC and the set of allowed addresses, forwarded from ECC-PRE-SC.
- b) *allowedSender()*: It checks if the caller is allowed to interact with the contract. This function iterates through the list of allowed addresses to determine if the caller is authorized. This function is called privately inside the increment function.
- c) *increment(sender)*: It first verifies if the call originates from ECC-PRE-SC; otherwise, it rejects the request. If valid, it checks whether the sender's address is in the allowed list by computing its hash. If verified, it increments the count for that *SP*; otherwise, it rejects the request. Since separate ECC-PRE-SC and Count-SC instances are deployed per content and *SP*, the function tracks the count for the specific *SP* handling the request.
- d) *getCount()*: It returns the count of the content consumed or downloaded with respect to each *SP*.

E. Decryption Phase

There are two decrypt algorithms, Decrypt, which can be executed by the *CO* as it has encrypted it, and another ReDecrypt, which can be executed by the end user, where the user uses its own private key to decrypt *C'* and get k_i . k_i is further used to decrypt the encrypted content.

The user executes the following functions to access the encrypted content and decrypt the content: *reqKeyContent()*, *decryptKey()*, and *decryptContent()*.

- a) *reqKeyContent()*: This function basically requests *C'* and C_m with *SP*.
- b) *decryptKey(C', id_{CO}, pk_{CO}, id_{U_i}, sk_{U_i}, params)*: *U* uses *C'*, *id_{CO}*, *pk_{CO}*, *id_{U_i}*, *sk_{U_i}*, and *params* to decrypt *C'*, using the *decryptKey()* function, and returns k_i , which will be used to decrypt encrypted content.
- c) *decryptContent(C_m, k_i)*: *U* uses C_m , and k_i , to decrypt C_m , using the *decryptContent()* function, and returns the original content *m* to the user.

The two algorithms are as follows:

Decrypt: While inputting *id_{CO}*, *C*, *sk_{CO}*, and *params*, *CO* can operate as follows:

- a) If $C_5P = C_4 + H_3(C_1, C_2, C_3, C_4)C_2$, then proceed; otherwise, output $\perp$.
- b) Compute $(k_i || \delta) = C_3 \oplus H_2((sk_{CO})C_2)$.
- c) If $C_2 = c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1}$, where $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$, the algorithm returns k_i as the key to the content. Otherwise, it outputs $\perp$, indicating a decryption failure.

Re-Decrypt: While inputting *id_{CO}*, *C'*, *pk_{CO}*, *id_{U_i}* and *sk_{U_i}*, and *params*, the user *U_i* operates as follows:

- a) Compute $s' = H_4(id_{CO}, id_{U_i}, (sk_{U_i})P)$.
- b) Compute $(k_i || \delta) = C'_3 \oplus H_2(s'(C'_1 + C'_2))$.
- c) If $C'_4 = (s')^{-1}rP(pk_{CO1} + pk_{CO2})$, where $r = H_1(m, \delta, id_{CO}, pk_{CO})$, the algorithm returns k_i as the content key. Otherwise, it outputs $\perp$, indicating a decryption failure.

VI. CORRECTNESS AND VALIDATION OF THE SCHEME

A. Correctness of our Proxy Re-encryption Scheme

The proposed ECC based Proxy Re-encryption scheme is correct. There are two defined decryption algorithms, Decrypt and Re-Decrypt. To show the Decrypt correctly gives out the content key, we firstly verify,

$$C_5P = C_4 + H_3(C_1, C_2, C_3, C_4)C_2$$

Now, we take the left hand side of the equation,

$$\begin{aligned} C_5 \cdot P &= (t + c \cdot r \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1} \cdot H_3(C_1, C_2, C_3, C_4)) \cdot P \\ &= t \cdot P + c \cdot r \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1} \cdot H_3(C_1, C_2, C_3, C_4) \cdot P \\ &= C_4 + C_2 \cdot H_3(C_1, C_2, C_3, C_4) \end{aligned}$$

After the verification process, the Decrypt algorithm is checked, Here we take,

$$\begin{aligned} C_3 \oplus H_2((sk_{CO})C_2) &= C_3 \oplus H_2(sk_{CO} \cdot (c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2}) \cdot sk_{CO}^{-1})) \\ &= C_3 \oplus H_2(c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2})) \\ &= C_3 \oplus H_2(f \cdot P \cdot (pk_{CO1} + pk_{CO2})) \\ &= C_3 \oplus H_2(f \cdot Q_A) \\ &= (k_i || \delta) \oplus H_2(f \cdot Q_A) \oplus H_2(f \cdot Q_A) \\ &= (k_i || \delta) \end{aligned}$$

Hence, the Decrypt algorithm correctly outputs the content key, k_i .

Similar to above for the Re-Decrypt algorithm, we first take

$$\begin{aligned} s' &= H_4(id_{CO}, id_{U_i}, (sk_{U_i})P) \\ &= H_4(id_{CO}, id_{U_i}, (sk_{U_i} \cdot P)) \\ &= H_4(id_{CO}, id_{U_i}, pk_{U1}) \\ &= s \end{aligned}$$

After above verification, we take the Re-Decrypt algorithm,

$$\begin{aligned} C'_3 \oplus H_2(s'(C'_1 + C'_2)) &= C'_3 \oplus H_2(s((rk_1 \cdot C_1) + (rk_2 \cdot C_1))) \\ &= C'_3 \oplus H_2(s((rk_1 \cdot r \cdot P) + (rk_2 \cdot r \cdot P))) \\ &= C'_3 \oplus H_2(s(((s^{-1} \cdot c \cdot pk_{CO1}) \cdot r \cdot P) + ((s^{-1} \cdot c \cdot pk_{CO2}) \cdot r \cdot P))) \\ &= C'_3 \oplus H_2(((c \cdot pk_{CO1}) \cdot r \cdot P) + ((c \cdot pk_{CO2}) \cdot r \cdot P)) \\ &= C'_3 \oplus H_2(c \cdot r \cdot P(pk_{CO1} + pk_{CO2})) \\ &= C_3 \oplus H_2(c \cdot r \cdot Q_A) \\ &= (k_i || \delta) \oplus H_2(f \cdot Q_A) \oplus H_2(f \cdot Q_A) \\ &= (k_i || \delta) \end{aligned}$$

Here, the delegatee will output the correct content key, k_i .

In summary, the proposed ECC-based PRE scheme is correct.

B. Validation of Smart Contract

We used a collection of security validation tools to check the vulnerability and weak points in our smart contracts. The list of tools that we use to analyze our smart contracts are listed in Table II. Using them, we identified bunch of vulnerabilities with its implication and severity level. We implemented the fix regarding the vulnerabilities. The identified vulnerabilities are as follows:

- 1) **Missing Zero Address Validation:** The PRE contract lacked validation for the zero address in its constructor when receiving the address of the Counter contract, as identified by Slither [42]. While the contract currently instantiates Counter internally, future modifications allowing an external Counter address could introduce a

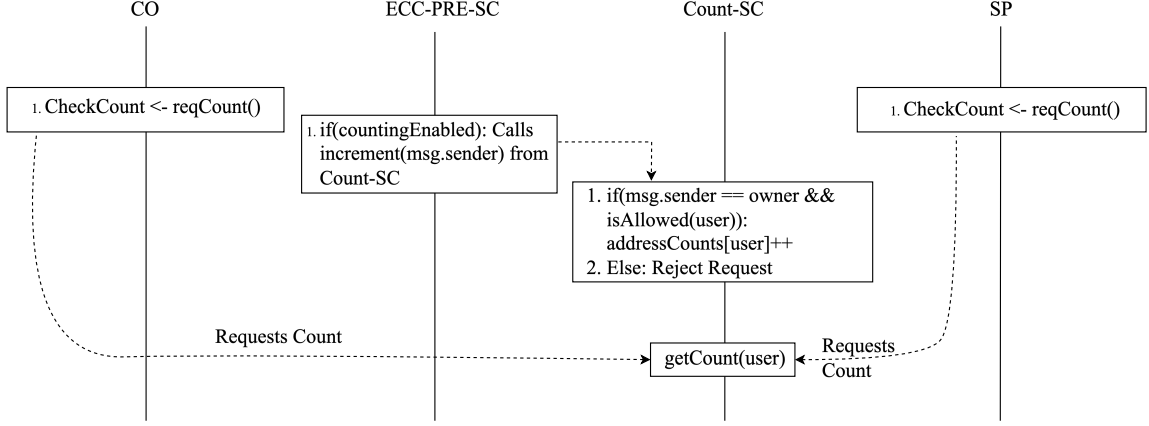


Fig. 3: Count SC Process

TABLE II: Tools used for our Smart Contract Analysis

Tool	Description
semgrep [41]	A static analysis tool that can be used to identify potential security vulnerabilities in code. It uses pattern matching to find common coding errors.
slither-0.10.0 [42]	A static analysis framework for Solidity. It performs analyses on the code to detect vulnerabilities and provides comprehensive reports.
solhint-3.3.8 [43]	A linting tool for Solidity that helps enforce code style guidelines and identify potential errors.
smartcheck [44]	A static analysis tool designed specifically for Solidity. It checks for common security vulnerabilities and code quality issues.
maian [45]	A static analysis tool for Ethereum smart contracts that focuses on detecting security vulnerabilities.
mythril-0.24.7 [46]	A security analysis tool for Ethereum smart contracts. It uses symbolic execution and taint analysis to detect vulnerabilities.
securify [47]	A static analysis tool that formally verifies the security properties of Ethereum smart contracts. It uses a theorem prover to ensure the code adheres to specified security rules.

risk where a malicious actor supplies the zero address, rendering the access control mechanism ineffective. This medium-severity issue has been mitigated by implementing a `require` statement in the Counter contract's constructor, ensuring that the `owner` address is not zero.

- 2) **Uncached Array Length:** Tools like Semgrep [41] and Slither flagged repeated access to `allowedAddresses.length` in the Counter contract's constructor loop as an inefficiency. Since accessing storage repeatedly increases gas costs, we optimized this by caching the array length in a local variable `allowedAddressesLength` before the loop, reducing redundant reads and improving efficiency. Though this issue was of low severity, this optimization enhances contract performance.
- 3) **Unoptimized Boolean Equality:** The `allowedSender` function in the Counter contract performed explicit boolean equality checks (e.g., `if (allowedAddresses[i] == hash)`), which Slither identified as a minor gas inefficiency. To optimize this, we removed explicit comparisons and directly returned `allowedAddressHashes[hash]`,

simplifying the code while reducing gas costs.

- 4) **Immutable State Variables:** Slither identified that several state variables in the PRE contract — `c1`, `c1_y`, `c2`, `c2_y`, `c3`, `c4`, `c4_y`, `c5p`, `parity1`, `parity2`, `parity4`, and `hash` — are assigned values only once in the constructor and never modified. Declaring these variables as `immutable` optimizes storage access and reduces contract deployment costs. This simple modification makes the PRE contract more efficient and cost-effective.

The Maian tool [45] was used to analyze four smart contracts (Counter, PRE, FastECMul Library, and ECCLibrary) for SUICIDAL, PRODIGAL, and GREEDY vulnerabilities. The results confirmed that **none of the contracts were vulnerable**, as they contained no SUICIDE instructions, no CALL/SUICIDE instances, and could not receive Ether, eliminating lock vulnerabilities. Additional analysis using Mythril [46] and Securify [47] also reported no issues, reinforcing the security of the code.

Further improvements were made using Slither and Solhint [43], aligning naming conventions with Solidity Style Guidelines for better readability and maintainability. SmartCheck [44] flagged some private-to-public visibility changes, but these were false positives that would have compromised security.

By implementing these fixes, we enhanced both security and efficiency. The zero address check mitigates a potential vulnerability, while boolean equality and array length optimizations reduce gas costs, ensuring a robust and cost-effective privacy-preserving re-encryption solution.

VII. SECURITY ANALYSIS

In this section, we analyse and show that our proposed system model is secure against the threat model defined in Section 4.2. Additionally, a summary Table III has been provided to map threats, defenses, formal result and Smart Contract Component.

A. Analyzing multiple re-encrypted keys

Theorem 1. Assume the adversary $\mathcal{A}$ has access to multiple re-encrypted content keys C' on the blockchain and wants to

TABLE III: Summary Threat Model: Elements, Mitigations, and Proof References

Threat	Adversary & Goal	Mitigation in Design	Location of Proof
TM-1: Multi-sample analysis	External observer tries to derive the <i>content key</i> from multiple re-encrypted keys on-chain.	ECC-based Proxy Re-Encryption used, built over ECDLP/CDH; Hash H_2 hides $(k_i \delta)$; re-encryption never exposes sk_{CO} or k_i .	Sec. VII-A
TM-2: Count manipulation	CO/SP/an attacker attempts to change the content-consumption count.	Access control: only ECC-PRE-SC can call increment; sender must match hashed allowlist; on-chain immutability prevents code tampering.	Sec. VII-B
TM-3: Key reuse across contents	A legitimate user reuses k_1 (for content m) to decrypt another content m' .	Design goal enforces different k_i per content and user; decryption correctness requires matching ciphertext-key pair only.	Sec. VII-C
TM-4: Honest-but-curious SP	SP observes requests/transactions to infer the content key.	Security rests on ECDLP/CDH; SP never gets plaintext content-key; zero-knowledge proof (ZKP) gates service without disclosure.	Sec. VII-D
TM-5: SP impersonation / DoS	Adversary fakes SP to obtain C' or floods PRE contract.	ZKP blocks impersonation; ECC-PRE-SC answers only to valid proofs, reducing attack surface for request floods.	Sec. VII-E

TM: Threat Model

derive the original content key k_i . Additionally, assuming $\mathcal{A}$ has no access to H_2 hash function. The probability that a PPT adversary $\mathcal{A}$ obtains content key k_i by analyzing multiple re-encrypted key is negligible.

Proof. The re-encrypted content key C' on the blockchain is secured by an elliptic curve cryptography-based proxy re-encryption scheme. Its security relies on the intractability of the Elliptic Curve Discrete Logarithm Problem (ECDLP). As long as ECDLP remains hard, deriving the original content key k_i from C' , even with multiple instances, is computationally infeasible.

Let the adversary know the structure of $C' = (C'_1, C'_2, C'_3, C'_4)$, where

$$C'_1 = rk_1 \cdot C_1, C'_2 = rk_2 \cdot C_1, C'_3 = C_3, C'_4 = rk_3 \cdot C_1$$

and $rk_{CO \rightarrow U_i} = (rk_1, rk_2, rk_3)$ is the re-encryption key.

To obtain the original content key k_i from C' , the adversary needs to solve the following equation:

$$\begin{aligned} (k_i||\delta) &= C_3 \oplus H_2(f \cdot Q_A) \\ &\text{or} \\ (k_i||\delta) &= C'_3 \oplus H_2(sk_{CO} \cdot C_2) \\ &\text{or} \\ (k_i||\delta) &= C'_3 \oplus H_2(C'_1 + C'_2) \end{aligned}$$

To solve this equation, the adversary needs to:

- 1) Obtain f , which is $c \cdot r$ where $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$ and c is an ephemeral randomness. Here, to obtain the value of r , $\mathcal{A}$ needs to know the k_i and δ , which are unknown.
- 2) Obtain sk_{CO} which is private to the entity CO only and infeasible to compute with the information that $\mathcal{A}$ has. Additionally, provided that $\mathcal{A}$ has C'_2 , to calculate $C_2 = (rk_2)^{-1} \cdot C'_2$, first, rk_2 is unknown, and second, even if rk_2 were known, it would be computationally infeasible to extract C_2 due to the hardness of the ECDLP.
- 3) As mentioned in the theorem, $\mathcal{A}$ does not have access to the H_2 hash function, so $\mathcal{A}$ cannot compute $(k_i||\delta)$. Furthermore, the value of δ is unknown, making it infeasible to compute k_i .

Provided the above proof, we can conclude that our system is safe against obtaining a content key by analyzing the multiple re-encrypted keys. $\square$

B. Manipulation of Counting in Blockchain

Theorem 2. Consider the Increment function in the Count-SC. The probability that a PPT adversary $\mathcal{A}$ manipulates the count using increment function through the Count-SC is negligible.

Proof. Let Count-SC be the counting smart contract, and ECC-PRE-SC be the re-encryption smart contract with $addr(Count-SC)$ and $addr(ECC-PRE-SC)$ being their respective addresses. During the deployment, an instance of Count-SC is also deployed by keeping $addr(ECC-PRE-SC)$ as the owner of Count-SC. For a CO , SP or an $\mathcal{A}$ to manipulate the count, they would need to consider following scenarios:

- 1) *Tamper Blockchain Ledger:* Modify the Count-SC code, which is immutable on the blockchain and computationally infeasible due to the blockchain's tamper-resistance.
- 2) *Bypass Access Control:* Attempt to directly call the increment function of the Count-SC, regardless of whether they know $addr(Count-SC)$. This is infeasible because the Count-SC is deployed with the $addr(ECC-PRE-SC)$ embedded as its sole owner, enforcing an access control mechanism where only requests originating from $addr(ECC-PRE-SC)$ can trigger the increment function. Any attempt by a valid SP, the CO, or any other adversary to directly call the increment function will be rejected.
- 3) *Perform ZKP for Re-Encryption following Count Increment:* To increment the count legitimately, $\mathcal{A}$ must pass a Zero-Knowledge Proof (ZKP) verification and re-encryption check before calling the increment function. Only a valid SP can pass the ZKP, triggering to re-encryption and subsequently invoking the increment function of Count-SC with $addr(ECC-PRE-SC)$. Without ZKP verification and re-encryption key check, $\mathcal{A}$ request will be rejected.

Since all potential methods for manipulating the count are impractical, the probability of an adversary succeeding in manipulating the count in blockchain is negligible. $\square$

C. Leveraging One Content's Key to Decrypt Another Content

Theorem 3. Assume a malicious user obtains a content key k_1 for encrypted content C_m from a legitimate process and also has obtained an encrypted content (C'_m). The probability that a PPT adversary $\mathcal{A}$ decrypting C'_m with k_1 is negligible.

Proof. Let k_1 be the key used to encrypt content m and produce C_m , and k_2 be the key used to encrypt a different content m' to produce $C_{m'}$. Even if a user obtains k_1 legitimately, they cannot decrypt $C_{m'}$ because:

$$\text{Dec}(C_{m'}, k_1) \neq m_2$$

where m_2 is the plaintext of $C_{m'}$

This is due to the key independence property of the encryption scheme, which ensures that different keys are used for different content. Hence, this proves that our system is resistant to leveraging one content's key to decrypt another content. $\square$

D. SP acting as honest but curious entity

Theorem 4. Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary that acts as an honest but curious SP. The probability that $\mathcal{A}$ can obtain the original content key k_i from the transactions it handles is negligible.

Proof. Let the CO generate a key pair (sk_{CO}, pk_{CO}) and encrypt the content key k_i with pk_{CO} , producing the encrypted key $C = (C_1, C_2, C_3, C_4, C_5)$. Similarly, the User (U) generates a key pair (sk_U, pk_U) . The SP, acting as an honest-but-curious adversary $\mathcal{A}$, generates the re-encryption key: $rk_{CO \rightarrow U} = \text{reKeyGenerate}(c, params, id_{CO}, id_U, pk_{CO}, pk_U)$

The SP sends $rk_{CO \rightarrow U}$ to the ECC-PRE-SC and receives a re-encrypted content key, forwarding it to the User. To obtain k_i , the adversary $\mathcal{A}$ must:

- 1) Compute sk_{CO} or sk_U from the public keys pk_{CO} and pk_U , respectively, and then use the Decrypt or Re-Encrypt algorithms to obtain k_i . However, this would violate the hardness assumption of the ECDLP.
- 2) Compute the value of $c \cdot r \cdot P \cdot (pk_{CO1} + pk_{CO2})$ from the re-encrypted content key C' , where $r = H_1(k_i, \delta, id_{CO}, pk_{CO})$. However, this would violate the hardness assumption of the CDH problem.

Since ECDLP and CDH are computationally infeasible, the probability of $\mathcal{A}$ obtaining k_i is negligible. Thus, the scheme is secure against Threat Model 4, where the SP attempts to extract the content key. $\square$

E. Security against malicious SP

Theorem 5. Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary that acts as a valid SP. The probability that $\mathcal{A}$ can impersonate a valid SP or disrupt the ECC-PRE-SC is negligible.

Proof. For $\mathcal{A}$ to successfully impersonate SP, $\mathcal{A}$ must forge a valid proof and challenge that satisfies the verification function in ECC-PRE-SC. The generation of proofs requires the knowledge of a secret number v and a random number j , which is only known to the valid SP.

Without knowing v and j , the probability of $\mathcal{A}$ generating a valid proof is:

$$\Pr[\mathcal{A}(params) = (i, o, yy, zz, A, B, \gamma, \alpha) \mid \text{verifyProof}(i, o, yy, zz, A, B, \gamma, \alpha) = \text{true}] = \frac{1}{2^{v \cdot j}}$$

As v and j increases, the probability of $\mathcal{A}$ forging a proof becomes negligible.

For $\mathcal{A}$ to disrupt the system, $\mathcal{A}$ must overload ECC-PRE-SC with requests. However, ECC-PRE-SC only responds to requests with valid proofs. Thus, the probability of $\mathcal{A}$ disrupting the system becomes negligible as given by the equation above. The ZKP mechanism ensures that only valid SPs can request the re-encrypted content key and that disrupting the system's functionality is computationally infeasible. $\square$

VIII. IMPLEMENTATION AND EVALUATION

In this section, we discuss the specifics of the implementation and evaluation of our scheme. Specifically, we evaluate the implementation of ECC-PRE-SC, the re-encryption smart contract, and Count-SC, the counting smart contract.

A. Simulation Specifics

The SECP256k1 elliptic curve was used for all elliptic curve point operations. Our program was written and compiled using Solidity 0.7.26 and the elliptic curve arithmetic operations are implemented with the elliptic-curve-solidity and FastEcMul library. We used the Sepolia Blockchain Network and the Alchemy API to connect to the blockchain instance to simulate our scheme. The deployment and interaction were performed using Python on a computer with an Intel i7-1355U running Windows 11.

B. Performance Analysis

For the following analysis, we let $l = 128$ bits, where l is the length of the key which is used to encrypt the content. In our case, the plaintext data that will be encrypted is a content key, so we let $n = 128, 192$, and 256 bits, respectively, representing various length keys based on key size recommendations for symmetric key lengths for the AES encryption algorithm [48]. As shown in Fig. 4, the size of the message is largely irrelevant in terms of overall gas consumption. The principal driver of gas consumption is the execution of the four elliptic curve scalar multiplication operations (with a 256-bit scalar) and one elliptic curve point addition operation that are all needed to verify and compute the re-encrypted ciphertexts in the Re-Encrypt function. Overall, the ECC-PRE-SC has a consistent gas cost with little variance regardless of the actual message

length, so our scheme has great flexibility in the size of the desired re-encrypted message. The counting contract Count-SC consumes far less gas because it must only worry about keeping a count. Retrieving the current count for a specific *SP* costs around 24000 gas in the Sepolia Testnet, whereas as per our verification, the transaction cost for the increment function is not available due to internal transactions between the two contracts. This limitation arises because the increment() function is invoked as an internal transaction from the ECC-PRE-SC to the Count-SC, and current Ethereum testnet tools do not expose internal transaction gas costs directly. In future work, this cost could be estimated using dedicated benchmarking frameworks such as Hardhat’s gas reporter, Ganache simulations, or custom Solidity profiling tools that isolate internal calls. This would provide a more granular measurement and enhance reproducibility of our results.

While direct, standardized comparisons across privacy-preserving Ethereum contracts are not available, indicative gas-cost ranges from prior ZKP-based systems provide useful context. Our ECC-PRE-SC implementation on Sepolia incurs 1.66 M gas per transaction, which includes both elliptic-curve proxy re-encryption and embedded zero-knowledge verification. By contrast, previously reported values for contracts focused solely on ZKP primitives range from about 1 M gas for a simple zk-SNARK verification (e.g., zkay [49]) to 7.19 M gas for Bulletproof-based transfers (e.g., Zether [50]). These figures suggest that, even after incorporating additional ECC operations for proxy re-encryption, our design operates within a competitive cost envelope—substantially lower than Bulletproof-based schemes and only modestly above a minimal zk-SNARK verification, while providing stronger functionality. The tight integration of ZKP verification with proxy re-encryption, rather than invoking separate heavy proof-verification contracts, enables our approach to preserve privacy and enforce tamper-proof counting with practical on-chain efficiency, supporting feasible throughput under Ethereum’s 15–30 M gas block constraints without sacrificing verifiable content access tracking.

Table IV summarizes the measured performance of our ECC-PRE-SC and Count-SC deployments on the Sepolia testnet. The smart-contract re-encryption consumes 1,660,599 gas per transaction with negligible variability (variance $\leq 0.5\%$), demonstrating stable gas consumption under repeated invocations. The end-to-end latency, inclusive of Remote Procedure Call (RPC) overhead, averages 10.52 s with 50th, 90th, and 99th percentiles of 11.88 s, 12.10 s, and 12.15 s, respectively. The sustained throughput of 0.095 tx/s is primarily constrained by single-threaded submission and orchestration delays in the Python client, reflecting realistic deployment conditions under RPC latency and mempool contention rather than intrinsic blockchain limits. Off-chain cryptographic operations incur minimal cost: rekey generation averages 18.9 ms (95th percentile: 21.85 ms), a single elliptic-curve operation under SECP256k1 requires 5.84 ms, and ciphertext re-decryption using a user’s private key averages 39.65 ms. These measurements confirm that cryptographic computation overhead is negligible compared to network delays and on-chain transaction processing.

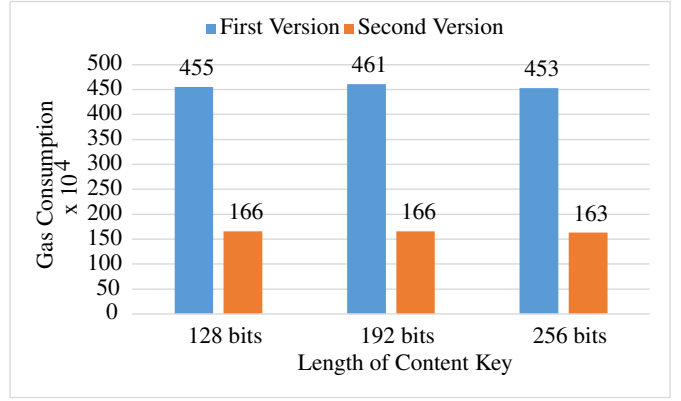


Fig. 4: Gas consumption comparison between the first and second versions of ECC-PRE-SC for varying content key lengths (128, 192, and 256 bits) on Sepolia.

C. Performance Improvement

During implementation, we observed that the initial version of the re-encryption scheme consumed significantly more gas due to multiple ECC-based operations, including multiplications and additions. The multiplication operation, being the most expensive, prompted us to identify a different library that provides speed-ups for ECC multiplication using window Non-Adjacent Form (wNAF) scalar representation and scalar decomposition through endomorphism [51]. Specifically, the gas consumption decreased from 455×10^4 to 166×10^4 for a 128-bit content key length, from 461×10^4 to 166×10^4 for 192 bits, and from 453×10^4 to 163×10^4 for 256 bits. This results in an average gas consumption reduction of approximately 63.84%. The comparison of the gas consumption for our first version of the system and the second improved version has been shown in the Fig. 4.

D. Lessons Learned

During the optimization and validation of our smart contracts, several important lessons were identified that may benefit future implementations. First, repeated access to storage variables, such as array lengths, should be avoided by caching values locally to reduce gas consumption. Second, employing optimized elliptic curve libraries that utilize techniques such as window Non-Adjacent Form (wNAF) scalar representation and scalar decomposition through endomorphism can significantly lower the cost of elliptic curve operations. Third, declaring immutable state variables when their values are set only once reduces storage overhead and deployment costs. Finally, validation checks, such as preventing zero-address assignments, are essential for ensuring contract robustness. Collectively, these practices improved both the efficiency and security of our contracts and can serve as practical guidance for developers aiming to write gas-efficient Solidity code.

IX. CONCLUSION

In this paper, we proposed a blockchain-based architecture designed to ensure the integrity and transparency of online

TABLE IV: Measured on-chain and off-chain performance of our ECC-PRE-SC and Count-SC implementation on Sepolia

Position	Metric	Value	Remarks
On-chain	Gas/tx (re-encrypt)	1,660,599 gas	Average over 30 smart contract re-encryption requests.
On-chain	End-to-end latency (avg / p50 / p90 / p99)	10521 / 11877 / 12101 / 12151 ms	Measured from client request to final response, including RPC delays.
On-chain	Measured throughput	0.095 tx/s	Single-threaded submission; reflects RPC latency and mempool contention.
Off-chain	rekey_generate (avg / p50 / p95)	18.907 / 18.659 / 21.852 ms	Executed locally; cryptographic key generation overhead.
Off-chain	ec_ops (avg / p50 / p95)	5.840 / 5.702 / 6.918 ms	Single elliptic-curve operation timings under SECP256k1.
Off-chain	redencrypt (avg / p50 / p95)	39.648 / 39.365 / 45.173 ms	Re-decryption using user's private key; excludes network latency.

content consumption tracking. Our system addresses the limitations of centralized platforms and also decentralized counting solutions by introducing two interactive smart contracts: an Elliptic Curve Cryptography-based Proxy Re-Encryption Smart Contract (ECC-PRE-SC) and a dedicated Counting Smart Contract (Count-SC). The core contribution lies in the cryptographic enforcement of the consumption count; an increment is atomically recorded only as a direct consequence of a successful, Zero-Knowledge-Proof-verified re-encryption of a content key, thus making the count an unavoidable byproduct of granting access. We demonstrated the feasibility of this design through a successful implementation on the Sepolia testnet, which showed consistent gas consumption regardless of content key size and achieved a significant gas reduction of approximately 63% after optimization. We also provide a brief security analysis indicating resistance to double-counting, replay, and front-running attacks, including collusion between users and SPs, while preserving key confidentiality and access privacy. By providing an immutable and verifiable record of content access, our work establishes a secure and transparent framework for digital content distribution, benefiting creators, service providers, and users alike.

REFERENCES

- [1] S. Joshi, D. Davidson, and J. Son, "Ensuring integrity in online content usage and download counting with smart contracts," in *Proceedings of the 20th International Conference on Security and Privacy in Communication Networks*, 2024. [Online]. Available: <https://securecomm.eai-conferences.org/2024/accepted-papers/>
- [2] P. Falkowski-Gilski and T. Uhl, "Current trends in consumption of multimedia content using online streaming platforms: A user-centric survey," *Computer Science Review*, vol. 37, p. 100268, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013719302473>
- [3] H. Tabrizchi and M. Kuchaki Rafsanjani, "A survey on security challenges in cloud computing: issues, threats, and solutions," *The Journal of Supercomputing*, vol. 76, no. 12, pp. 9493–9532, Dec. 2020. [Online]. Available: <https://doi.org/10.1007/s11227-020-03213-1>
- [4] A. Tatar, M. D. de Amorim, S. Fdida, and P. Antoniadis, "A survey on predicting the popularity of web content," *Journal of Internet Services and Applications*, vol. 5, no. 1, p. 8, Aug. 2014. [Online]. Available: <https://doi.org/10.1186/s13174-014-0008-y>
- [5] R. Xu, L. Zhang, H. Zhao, and Y. Peng, "Design of network media's digital rights management scheme based on blockchain technology," in *2017 IEEE 13th International Symposium on Autonomous Decentralized System (ISADS)*, 2017, pp. 128–133.
- [6] M. Lee, H. S. Choi, D. Cho, and H. Lee, "Can digital consumption boost physical consumption? the effect of online music streaming on record sales," *Decision Support Systems*, vol. 135, p. 113337, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167923620300920>
- [7] T. Pellegrini, *Digital Rights Management – Technologies, Application Areas, and Governance*. Wiesbaden: Springer Fachmedien Wiesbaden, 2023, pp. 1–17. [Online]. Available: https://doi.org/10.1007/978-3-658-34048-3_79-2
- [8] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [9] A. Foroughi, M. Albin, and S. Gillard, "Digital rights management: a delicate balance between protection and accessibility," *Journal of Information Science*, vol. 28, no. 5, pp. 389–395, 2002. [Online]. Available: <https://doi.org/10.1177/016555150202800504>
- [10] D. KUNDUR and K. KARTHIK, "Video fingerprinting and encryption principles for digital rights management," *Proceedings of the IEEE*, vol. 92, no. 6, pp. 918–932, 2004.
- [11] W. J. Buchanan, "Nizk (non-interactive zero knowledge) proofs of discrete - log equality with powershell," 2025. [Online]. Available: <https://asecuritysite.com/powershell/chaum>
- [12] IBM, "Blockchain for supply chain solutions," 2025, accessed: Sep. 4, 2025. [Online]. Available: <https://www.ibm.com/solutions/blockchain-supply-chain>
- [13] M. Westerkamp, F. Victor, and A. Küpper, "Blockchain-based supply chain traceability: Token recipes model manufacturing processes," in *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2018, pp. 1595–1602.
- [14] H. Wu, S. Jiang, and J. Cao, "High-efficiency blockchain-based supply chain traceability," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 3748–3758, 2023.
- [15] Y. Ding, D. Luo, H. Xiang, W. Liu, and Y. Wang, "Design and implementation of blockchain-based digital advertising media promotion system," *Peer-to-Peer Networking and Applications*, vol. 14, no. 2, pp. 482–496, 2021. [Online]. Available: <https://doi.org/10.1007/s12083-020-00984-5>
- [16] B. Nassirzadeh, A. Heinle, S. Leonardos, A. Hasan, and V. Ganesh, "Countchain: A decentralized oracle network for counting systems," in *2024 IEEE International Conference on Blockchain (Blockchain)*, 2024, pp. 324–331.
- [17] V. Chouhan and A. Arora, "Blockchain-based secure and transparent election and vote counting mechanism using secret sharing scheme," *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, no. 10, pp. 14009–14027, Oct. 2023. [Online]. Available: <https://doi.org/10.1007/s12652-022-04108-0>
- [18] U. Jafar, M. J. A. Aziz, and Z. Shukur, "Blockchain for electronic voting system—review and open research challenges," *Sensors*, vol. 21, no. 17, 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/17/5874>
- [19] M. S. Ozdayi, M. Kantarcioglu, and B. Malin, "Leveraging blockchain for immutable logging and querying across multiple sites," *BMC Medical Genomics*, vol. 13, no. 7, p. 82, 2020.
- [20] X. Xiao, Y. Zhang, L. Y. Zhang, Z. Hua, Z. Liu, and J. Huang, "Faircms: Cloud media sharing with fair copyright protection," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 5, pp. 6192–6209, 2024.
- [21] T. Feng, D. Wang, and R. Gong, "A blockchain-based efficient and verifiable attribute-based proxy re-encryption cloud sharing

- scheme,” *Information*, vol. 14, no. 5, 2023. [Online]. Available: <https://www.mdpi.com/2078-2489/14/5/281>
- [22] L. Yan, H. Qin, K. Yang, H. Xie, X. A. Wang, and S. Liu, “Pairing-free certificate-based proxy re-encryption plus scheme for secure cloud data sharing,” *Electronics*, vol. 13, no. 3, 2024. [Online]. Available: <https://www.mdpi.com/2079-9292/13/3/534>
 - [23] F. Wang, J. Cui, Q. Zhang, D. He, and H. Zhong, “Blockchain-based secure cross-domain data sharing for edge-assisted industrial internet of things,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 3892–3905, 2024.
 - [24] Z. Zhang and L. Zhao, “A design of digital rights management mechanism based on blockchain technology,” in *Blockchain – ICBC 2018*, S. Chen, H. Wang, and L.-J. Zhang, Eds. Cham: Springer International Publishing, 2018, pp. 32–46.
 - [25] A. Garba, A. D. Dwivedi, M. Kamal, G. Srivastava, M. Tariq, M. A. Hasan, and Z. Chen, “A digital rights management system based on a scalable blockchain,” *Peer-to-Peer Networking and Applications*, vol. 14, no. 5, pp. 2665–2680, Sep. 2021. [Online]. Available: <https://doi.org/10.1007/s12083-020-01023-z>
 - [26] J. Gao, H. Yu, X. Zhu, and X. Li, “Blockchain-based digital rights management scheme via multiauthority ciphertext-policy attribute-based encryption and proxy re-encryption,” *IEEE Systems Journal*, vol. 15, no. 4, pp. 5233–5244, 2021.
 - [27] V. Y. Kemmoe, Y. Kwon, R. Hussain, S. Cho, and J. Son, “Leveraging smart contracts for secure and asynchronous group key exchange without trusted third party,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 3176–3193, 2023.
 - [28] P. Duan, Z. Ma, H. Gao, T. Tian, and Y. Zhang, “Multi-authority attribute-based encryption scheme with access delegation for cross blockchain data sharing,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 323–337, 2025.
 - [29] S. Hussain, I. Ullah, H. Khattak, M. Adnan, S. Kumari, S. S. Ullah, M. A. Khan, and S. J. Khattak, “A lightweight and formally secure certificate based signcryption with proxy re-encryption (cbsre) for internet of things enabled smart grid,” *IEEE Access*, vol. 8, pp. 93 230–93 248, 2020.
 - [30] K. O.-B. O. Agyekum, Q. Xia, E. B. Sifah, C. N. A. Cobblah, H. Xia, and J. Gao, “A proxy re-encryption approach to secure data sharing in the internet of things based on blockchain,” *IEEE Systems Journal*, vol. 16, no. 1, pp. 1685–1696, 2022.
 - [31] I. Keshta, Y. Aoudni, M. Sandhu, A. Singh, P. A. Xalikovich, A. Rizwan, M. Soni, and S. Lalar, “Blockchain aware proxy re-encryption algorithm-based data sharing scheme,” *Physical Communication*, vol. 58, p. 102048, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1874490723000514>
 - [32] P. Shabisha, A. Braeken, A. Touhafi, and K. Steenhaut, “Elliptic curve qu-vanstone based signcryption schemes with proxy re-encryption for secure cloud data storage,” in *Cloud Computing and Big Data: Technologies, Applications and Security*, M. Zbakh, M. Essaïdi, P. Manneback, and C. Rong, Eds. Cham: Springer International Publishing, 2019, pp. 1–18.
 - [33] A. Kiayias, A. Russell, B. David, and R. Oliynykov, “Ouroboros: A provably secure proof-of-stake blockchain protocol,” in *Advances in Cryptology – CRYPTO 2017*, J. Katz and H. Shacham, Eds. Cham: Springer International Publishing, 2017, pp. 357–388.
 - [34] K. Wüst and A. Gervais, “Do you need a blockchain?” in *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*, 2018, pp. 45–54.
 - [35] N. Radziwill, “Blockchain revolution: How the technology behind bitcoin is changing money, business, and the world,” *The Quality Management Journal*, vol. 25, no. 1, pp. 64–65, 2018, name - American Society for Quality; Penguin Random House; Copyright - Copyright Taylor & Francis Ltd. 2018; Last updated - 2023-12-04; SubjectsTermNotLitGenreText - New York; Ecuador; United States–US. [Online]. Available: <https://www.ezproxy.library.unlv.edu/login?url=https%3A%2F%2Fwww.proquest.com%2Fscholarly-journals%2Fblockchain-revolution-how-technology-behind%2Fdocview%2F2114612269%2Fse-2%3Faccountid%3D3611>
 - [36] S. Ølnes, “Beyond bitcoin enabling smart government using blockchain technology,” in *Electronic Government*, H. J. Scholl, O. Glassey, M. Janssen, B. Klievink, I. Lindgren, P. Parycek, E. Tambouris, M. A. Wimmer, T. Janowski, and D. Sá Soares, Eds. Cham: Springer International Publishing, 2016, pp. 253–264.
 - [37] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” pp. 1–32, 2014. [Online]. Available: <https://cryptodeep.ru/doc/paper.pdf>
 - [38] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger fabric: a distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys ’18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3190508.3190538>
 - [39] M. Lohava, G. Losa, D. Mazières, G. Hoare, N. Barry, E. Gafni, J. Jove, R. Malinowsky, and J. McCaleb, “Fast and secure global payments with stellar,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, ser. SOSP ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 80–96. [Online]. Available: <https://doi.org/10.1145/3341301.3359636>
 - [40] J.-C. Cheng, N.-Y. Lee, C. Chi, and Y.-H. Chen, “Blockchain and smart contract for digital certificate,” in *2018 IEEE International Conference on Applied System Invention (ICASI)*, 2018, pp. 1046–1051.
 - [41] “Decurity/semgrep-smart-contracts: Semgrep rules for smart contracts based on defi exploits,” 2024. [Online]. Available: <https://github.com/Decurity/semgrep-smart-contracts?tab=readme-ov-file>
 - [42] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.
 - [43] “Solhint,” 2017. [Online]. Available: <https://github.com/protofire/solhint>
 - [44] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, “Smartcheck: static analysis of ethereum smart contracts,” in *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*, ser. WETSEB ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 9–16. [Online]. Available: <https://doi.org/10.1145/3194113.3194115>
 - [45] I. Nikolić, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, “Finding the greedy, prodigal, and suicidal contracts at scale,” in *Proceedings of the 34th Annual Computer Security Applications Conference*, ser. ACSAC ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 653–663. [Online]. Available: <https://doi.org/10.1145/3274694.3274743>
 - [46] “Smashing ethereum smart contracts for fun and real profit,” 2018. [Online]. Available: <https://github.com/muellerberndt/smashing-smart-contracts/blob/master/smashing-smart-contracts-1of1.pdf>
 - [47] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, “Securify: Practical security analysis of smart contracts,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 67–82. [Online]. Available: <https://doi.org/10.1145/3243734.3243780>
 - [48] E. Barker and A. Roginsky, “Transitioning the use of cryptographic algorithms and key lengths,” 2019. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-131Ar2>
 - [49] S. Steffen, B. Bichsel, M. Gersbach, N. Melchior, P. Tsankov, and M. Vechev, “zkay: Specifying and enforcing data privacy in smart contracts,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1759–1776. [Online]. Available: <https://doi.org/10.1145/3319535.3363222>
 - [50] B. Bünz, S. Agrawal, M. Zamani, and D. Boneh, “Zether: Towards privacy in a smart contract world,” in *Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 423–443. [Online]. Available: https://doi.org/10.1007/978-3-030-51280-4_23
 - [51] “elliptic-curve-solidity,” 2019. [Online]. Available: <https://github.com/witnet/elliptic-curve-solidity/tree/master>